Designing a Thread Safe Banking System
Concurrency Control and Deadlock Prevention in Python

Introduction

A banking system that allows several users to interact with the same account at the same time introduces a set of problems that do not exist in a single threaded program. If two threads read a balance, calculate a new value and write it back without any coordination, one of those updates can be silently lost, which is the race condition that Nguyen Quan (2022, ch. 12) sets out as the starting point for any discussion of shared state in a multithreaded Python program. The assignment is to design a BankAccount class that remains correct under concurrent access, to simulate multiple users transacting on the same account, and to show that a transfer between two accounts can be carried out without the risk of deadlock. The solution is written in Python and relies on the threading module and its Lock primitive, together with a resource ordering strategy for the transfer operation, following the general pattern for safe concurrent code that Nguyen Quan (2022, ch. 12) and McDonald (2023, ch. 17) both work through in more detail.

Designing the BankAccount Class

The BankAccount class stores an account number, a private balance and a lock that belongs to that specific account. Every method that reads or changes the balance acquires the lock first, carries out its work, and releases the lock automatically through Python's context manager syntax. McDonald (2023, ch. 17) argues that this with statement style is the idiomatic way to manage a resource such as a lock in Python, since it guarantees the lock is released even if an exception is raised part way through the method, rather than relying on a programmer to remember to call release() manually. Using a lock scoped to each individual account rather than one global lock across the whole system matters here, since a single shared lock would force every account in the bank to queue behind every other account's transaction, which would defeat the purpose of allowing concurrent activity at all (Nguyen Quan, 2022, ch. 12). Deposits and withdrawals also validate their input before touching the lock, so a negative or zero amount is rejected early rather than being allowed to corrupt the balance.

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the available balance."""
    pass


class BankAccount:
    def __init__(self, account_number, balance=0):
        self.account_number = account_number
        self._balance = balance
        self._lock = threading.Lock()

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        with self._lock:
            self._balance += amount
```

```
    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive")
        with self._lock:
            if amount > self._balance:
                raise InsufficientFundsError(
                    f"Account {self.account_number} has insufficient funds"
                )
            self._balance -= amount

    def get_balance(self):
        with self._lock:
            return self._balance
```

Ensuring Thread Safety

Without a lock, the increase in deposit() would not happen as one atomic step. The interpreter has to read the current balance, add the amount, and store the result back, and a thread switch can occur at any point in that sequence. This is worth being precise about in Python specifically, since Nguyen Quan (2022, ch. 6, section 2) explains that the global interpreter lock only stops two threads from executing Python bytecode at literally the same instant, it does not stop the interpreter from switching between threads midway through an operation like balance = balance + amount, which is exactly where the lost update happens. Wrapping the increment in a Lock forces those three steps, the read, the addition and the write, to complete as a single unit before another thread is allowed in. Because the lock in this design lives on the instance rather than the class, two different BankAccount objects can be updated by two different threads at the same time with no contention between them, which keeps the system responsive as the number of accounts grows.

Preventing Deadlocks During Transfers

A transfer between two accounts is more delicate than a single deposit, because it needs to hold both accounts' locks at once so that the debit and the credit happen as one atomic operation. The danger is that if thread one locks account A and then waits for account B, while thread two has already locked account B and is waiting for account A, neither thread can proceed and the program hangs. Nguyen Quan (2022, ch. 12) describes this circular waiting as the core cause of deadlock in a multithreaded program and sets out lock ordering as the practical fix, removing the possibility of a cycle by making sure every thread requests shared locks in the same fixed sequence rather than whichever order happens to suit it. The transfer function below sorts the two accounts by their account number before locking them, so no matter which direction money is moving in, every thread in the system agrees on which lock to take first.

```
def transfer(from_account, to_account, amount):
    first, second = sorted(
        [from_account, to_account], key=lambda acc: acc.account_number
    )
    with first._lock:
        with second._lock:
            if amount > from_account._balance:
                raise InsufficientFundsError(
                    f"Account {from_account.account_number} has insufficient funds"
                )
```

```
            from_account._balance -= amount
            to_account._balance += amount
```

With this ordering in place, two simultaneous transfers moving money in opposite directions between the same pair of accounts will queue safely behind whichever thread locked the lower numbered account first, rather than deadlocking against each other.

Simulating Concurrent Transactions

The TransactionSimulator class represents several users acting on the same account at once. Each user is given a small list of deposit and withdraw instructions, and each user's instructions run on their own thread. A short random delay is added between operations to make the interleaving between threads less predictable, which is closer to how real users would behave than firing every operation at exactly the same instant.

```
class TransactionSimulator:
    def __init__(self, account):
        self.account = account

    def run_user_session(self, user_id, operations):
        for op, amount in operations:
            try:
                if op == "deposit":
                    self.account.deposit(amount)
                elif op == "withdraw":
                    self.account.withdraw(amount)
            except InsufficientFundsError as exc:
                print(f"User {user_id}: {exc}")
            time.sleep(random.uniform(0.001, 0.01))

    def simulate(self, users_operations):
        threads = []
        for user_id, operations in users_operations.items():
            thread = threading.Thread(
                target=self.run_user_session, args=(user_id, operations)
            )
            threads.append(thread)
            thread.start()
        for thread in threads:
            thread.join()


if __name__ == "__main__":
    account = BankAccount(account_number="ACC001", balance=1000)
    simulator = TransactionSimulator(account)

    users_operations = {
        "user_1": [("deposit", 100), ("withdraw", 50), ("deposit", 20)],
        "user_2": [("withdraw", 200), ("deposit", 150)],
        "user_3": [("deposit", 300), ("withdraw", 100)],
    }

    simulator.simulate(users_operations)
    print(f"Final balance: {account.get_balance()}")
```

Testing and Validation

Two kinds of tests matter for this system, and they are testing different things. The first group checks ordinary correctness on a single thread, for example that a deposit increases the balance by the expected amount and that a withdrawal larger than the balance raises InsufficientFundsError rather than silently going negative. McDonald (2023, ch. 20) frames this kind of test as the baseline that should exist regardless of concurrency, since a class that is not correct on a single thread has no chance of being correct under load either. The second group is the more important one for this assignment, since it checks that the class behaves correctly once concurrency is introduced. The concurrent deposit test starts one hundred threads that each deposit ten units into a fresh account and then checks that the final balance is exactly one thousand. If the lock were removed from deposit(), this test would fail intermittently, since some of the individual increments would be lost, which is precisely the kind of bug that only shows up under load and is easy to miss with single threaded testing alone. The deadlock test starts two threads that transfer money between the same two accounts in opposite directions and joins both threads with a timeout, a pattern McDonald (2023, ch. 20) recommends specifically so that a hanging test fails with a clear result instead of leaving the whole test run stuck. If the transfer function did not order its locks, this test would either hang or intermittently time out, whereas with ordered locking both threads complete promptly.

```
class TestBankAccount(unittest.TestCase):
    def test_deposit_increases_balance(self):
        account = BankAccount("TEST001", 100)
        account.deposit(50)
        self.assertEqual(account.get_balance(), 150)

    def test_withdraw_decreases_balance(self):
        account = BankAccount("TEST002", 100)
        account.withdraw(40)
        self.assertEqual(account.get_balance(), 60)

    def test_withdraw_insufficient_funds_raises(self):
        account = BankAccount("TEST003", 50)
        with self.assertRaises(InsufficientFundsError):
            account.withdraw(100)

    def test_concurrent_deposits_are_thread_safe(self):
        account = BankAccount("TEST004", 0)
        threads = [
            threading.Thread(target=account.deposit, args=(10,))
            for _ in range(100)
        ]
        for thread in threads:
            thread.start()
        for thread in threads:
            thread.join()
        self.assertEqual(account.get_balance(), 1000)

    def test_concurrent_transfer_does_not_deadlock(self):
        account_a = BankAccount("A", 500)
        account_b = BankAccount("B", 500)

        thread_1 = threading.Thread(target=transfer, args=(account_a, account_b, 100))
        thread_2 = threading.Thread(target=transfer, args=(account_b, account_a, 50))
        thread_1.start()
        thread_2.start()
```

```
        thread_1.join(timeout=5)
        thread_2.join(timeout=5)

        self.assertFalse(thread_1.is_alive())
        self.assertFalse(thread_2.is_alive())
        self.assertEqual(
            account_a.get_balance() + account_b.get_balance(), 1000
        )


if __name__ == "__main__":
    unittest.main()
```

Evaluation

The design meets the brief in that every public method on BankAccount is protected by a lock, transfers cannot deadlock because of the fixed lock ordering, and the test suite demonstrates both properties rather than just asserting them in comments. There are trade offs worth being honest about. Python's global interpreter lock already prevents two threads from executing Python bytecode at the exact same instant on a single process, which means some of the race conditions in this assignment would be less likely to surface than they would in a language without that restriction. This does not make the explicit locking unnecessary, since Nguyen Quan (2022, ch. 6, section 2) is clear that the GIL can still be released between bytecode instructions, and the transfer function genuinely needs to hold two locks across multiple statements regardless of what the GIL is doing at any given moment. A second limitation is that acquiring two locks per transfer, even in a fixed order, adds a small amount of contention compared with a design that avoided cross account locking altogether, for instance by using a single queue of transaction requests processed by one worker thread. Nguyen Quan (2022, ch. 14) discusses this kind of alternative under the umbrella of message passing between threads rather than shared state, and notes that it removes the deadlock risk entirely at the cost of turning every transfer into a serial operation. Given the scale described in the brief, the lock based approach used here is a reasonable balance between safety and throughput, though the queue based alternative would be worth revisiting if the number of accounts and the frequency of transfers grew substantially.

Conclusion

This assignment has produced a BankAccount class that protects its balance with a per instance lock, a transfer function that avoids deadlock through consistent lock ordering, a TransactionSimulator that exercises the system with concurrent users, and a test suite that specifically targets the race condition and deadlock scenarios the brief asks for. Together these pieces show that thread safety in a system like this is less about adding locks everywhere and more about being deliberate over which operations need to be atomic and in what order shared resources are acquired.

References

McDonald, J.C. (2023) Dead Simple Python: Idiomatic Python for the Impatient Programmer. 1st edn. New York: No Starch Press.

Nguyen Quan (2022) Advanced Python Programming. 2nd edn. Birmingham: Packt Publishing.